

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №2 по дисциплине
‘ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА’

Вариант №5

Выполнил:
Студент группы Р3210
Давидюк А.Ю.

Преподаватель:
Рыбаков С.Д.

Цель работы

Изучить численные методы решения нелинейных уравнений и реализовать три из них средствами программирования. Понять их сходства и различия.

Ход работы

Часть 1. 1 Решение нелинейного уравнения

Вариант №5: $f(x) = -2,7x^3 - 1,48x^2 + 19,23x + 6,35$

1. Отделить корни заданного нелинейного уравнения графически

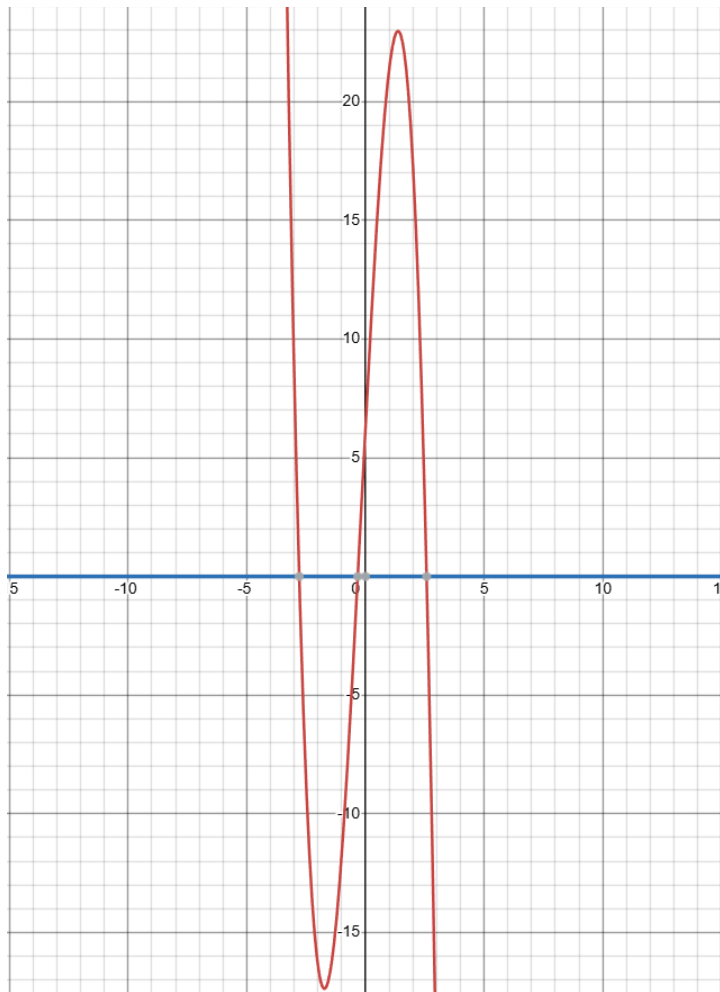


Рисунок 1 - График функции

2. Определить интервалы изоляции корней

Значения функции в контрольных точках:

	$f(x)$	Знак
-3	8,240	+
-2	-16,430	-
-1	-11,660	-
0	6,350	+
2	17,290	+

	$f(x)$	Знак
3	-22,180	–

Интервалы изоляции:

Корень	Интервал	Обоснование
Крайний левый	$[-3; -2]$	$f(-3) > 0,$ $f(-2) < 0$
Центральный	$[-1; 0]$	$f(-1) < 0,$ $f(0) > 0$
Крайний правый	$[2; 3]$	$f(2) > 0,$ $f(3) < 0$

3. Уточнить корни нелинейного уравнения (см. табл. 6) с точностью $\varepsilon=10^{-2}$
4. Используемые методы для уточнения каждого из 3-х корней многочлена представлены в таблице 7.
5. Вычисления оформить в виде таблиц (1-5), в зависимости от заданного метода. Для всех значений в таблице удерживать 3 знака после запятой.
- 5.5 Для метода простой итерации заполнить таблицу 5. Проверить условие сходимости метода на выбранном интервале.
6. Заполненные таблицы отобразить в отчете.

Расчетные формулы

1) Метод хорд (для правого корня, $[2; 3]$)

$$x_{k+1} = a_k - \frac{f(a_k) \cdot (b_k - a_k)}{f(b_k) - f(a_k)}$$

Правило выбора нового интервала:

- Если $f(a_k) \cdot f(x_{k+1}) < 0$, то $b_{k+1} = x_{k+1}$, $a_{k+1} = a_k$
- Если $f(x_{k+1}) \cdot f(b_k) < 0$, то $a_{k+1} = x_{k+1}$, $b_{k+1} = b_k$

Критерий окончания: $|x_{k+1} - x_k| < \varepsilon$

2) Метод простой итерации (для левого корня, $[-3; -2]$)

Преобразование уравнения к виду $x = \varphi(x)$

Итерационная формула: $x_{k+1} = \varphi(x_k)$

Критерий окончания: $|x_{k+1} - x_k| < \varepsilon$

3) Метод секущих (для центрального корня, $[-1; 0]$)

$$x_{k+1} = x_k - f(x_k) \cdot \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

Критерий окончания: $|x_{k+1} - x_k| < \varepsilon$

$$\varepsilon = 10^{-2}$$

№ шага	a	b	x	f(a)	f(b)	f(x)	$ x_{k+1} - x_k $
1	2,000	3,000	2,438	17,290	-22,180	4,215	-
2	2,438	3,000	2,528	4,215	-22,180	1,011	0,090
3	2,528	3,000	2,549	1,011	-22,180	0,238	0,021
4	2,549	3,000	2,554	0,238	-22,180	0,056	0,005

Таблица 1 - Уточнение правого корня методом хорд

№ шага	x_{k-1}	$f(x_{k-1})$	x_k	$f(x_k)$	x_{k+1}	$f(x_{k+1})$	$ x_k - x_{k+1} $
1	-0,500	-3,288	-0,200	2,466	-0,329	-1,248	0,129
2	-0,200	2,466	-0,329	-1,248	-0,305	-0,047	0,024
3	-0,329	-1,248	-0,305	-0,047	-0,304	0,002	0,001

Таблица 2 - Уточнение центрального корня методом секущих

№ шага	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
0	-2,500	-2,367	0,833	0,133
1	-2,367	-2,310	0,099	0,057
2	-2,310	-2,295	-0,016	0,015
3	-2,295	-2,299	0,003	0,004

Таблица 3 - Уточнение левого корня методом простой итерации

Проверка условия сходимости $|\phi'(x)| < 1$

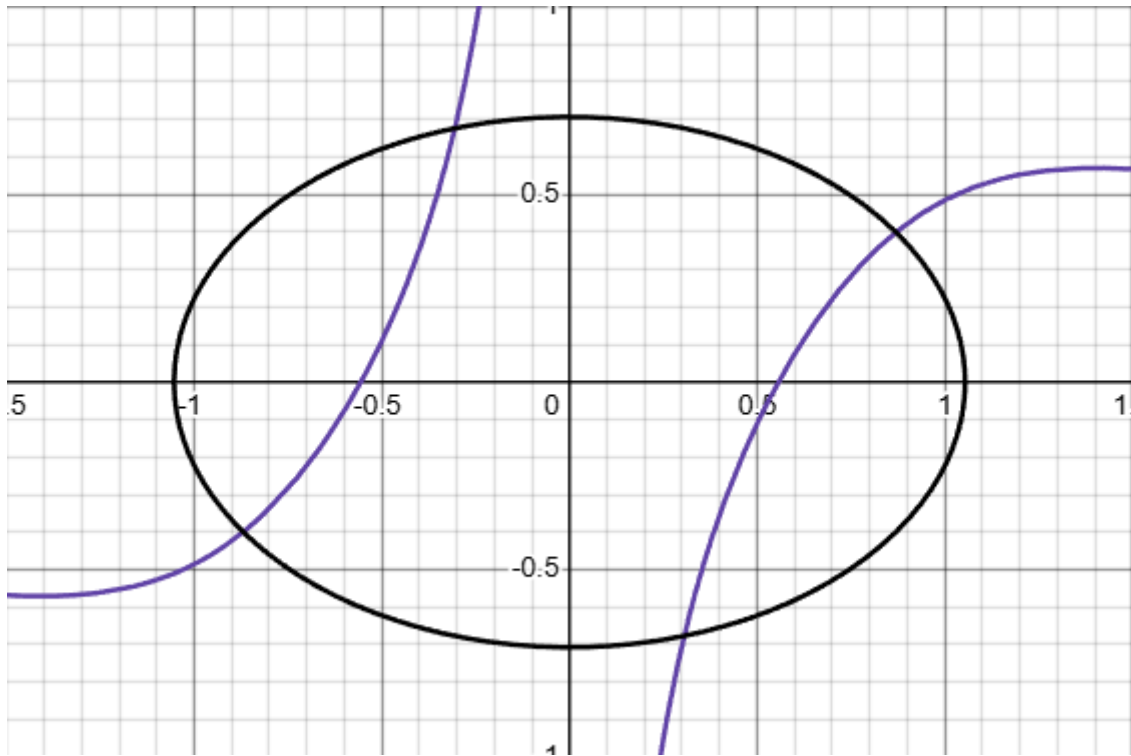
x	f'(x)	$\phi'(x)$	$ \phi'(x) < 1$
-3	-44,79	-0,792	Да
-2,5	-24,645	0,014	Да
-2	-7,25	0,71	Да

Часть 2. Решение системы нелинейных уравнений

Вариант №5: Метод Ньютона

$$\begin{cases} \tan(xy + 0,3) = x^2 \\ 0,9x^2 + 2y^2 = 1 \end{cases}$$

№ корня	Приближённые координаты	Интервал для x	Интервал для y
1	(0,3; -0,7)	[0; 0,6]	[-0,9; -0,5]
2	(-0,3; 0,7)	[-0,6; 0]	[0,5; 0,9]
3	(0,3; 0,7)	[0; 0,6]	[0,5; 0,9]
4	(-0,3; -0,7)	[-0,6; 0]	[-0,9; -0,5]



Рабочие формулы метода Ньютона для систем

Приведение системы к каноническому виду:

$$\begin{cases} F(x, y) = \operatorname{tg}(xy + 0,3) - x^2 = 0 \\ G(x, y) = 0,9x^2 + 2y^2 - 1 = 0 \end{cases}$$

Матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{\partial F}{\partial x} & \frac{\partial F}{\partial y} \\ \frac{\partial G}{\partial x} & \frac{\partial G}{\partial y} \end{pmatrix} = \begin{pmatrix} y \cdot \sec^2(xy + 0,3) - 2x & x \cdot \sec^2(xy + 0,3) \\ 1,8x & 4y \end{pmatrix}$$

где $\sec^2(\alpha) = 1 + \operatorname{tg}^2(\alpha)$.

Итерационная формула:

На каждом шаге решаем систему линейных уравнений:

$$J(x_k, y_k) \cdot \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} F(x_k, y_k) \\ G(x_k, y_k) \end{pmatrix}$$

Затем обновляем приближение:

$$x_{k+1} = x_k + \Delta x, y_{k+1} = y_k + \Delta y$$

Критерий окончания:

$$\sqrt{(\Delta x)^2 + (\Delta y)^2} < \varepsilon = 0,01$$

вычисления (на примере первого корня)

Начальное приближение: $x_0 = 0,6$, $y_0 = -0,5$ (четвёртый квадрант)

Итерация 1:

Шаг 1. Вычисление F , G :

$$\begin{aligned}xy + 0,3 &= 0,6 \cdot (-0,5) + 0,3 = 0 \\ \operatorname{tg}(0) &= 0 \\ F &= 0 - (0,6)^2 = -0,360 \\ G &= 0,9 \cdot (0,6)^2 + 2 \cdot (-0,5)^2 - 1 = 0,324 + 0,5 - 1 = -0,176\end{aligned}$$

Шаг 2. Вычисление элементов Якобиана:

$$\begin{aligned}\sec^2(0) &= 1 \\ \frac{\partial F}{\partial x} &= (-0,5) \cdot 1 - 2 \cdot 0,6 = -1,700 \\ \frac{\partial F}{\partial y} &= 0,6 \cdot 1 = 0,600 \\ \frac{\partial G}{\partial x} &= 1,8 \cdot 0,6 = 1,080 \\ \frac{\partial G}{\partial y} &= 4 \cdot (-0,5) = -2,000\end{aligned}$$

$$J = \begin{pmatrix} -1,700 & 0,600 \\ 1,080 & -2,000 \end{pmatrix}$$

Шаг 3. Решение системы $J \cdot [\Delta x, \Delta y]^T = -[F, G]^T$:

$$\begin{pmatrix} -1,700 & 0,600 \\ 1,080 & -2,000 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} 0,360 \\ 0,176 \end{pmatrix}$$

Решая методом Крамера или Гаусса:

$$\Delta x \approx -0,300, \Delta y \approx -0,250$$

Шаг 4. Новое приближение:

$$x_1 = 0,6 + (-0,300) = 0,300, y_1 = -0,5 + (-0,250) = -0,750$$

Шаг 5. Проверка точности:

$$\sqrt{(-0,300)^2 + (-0,250)^2} = \sqrt{0,1525} \approx 0,390 > 0,01 (\text{продолжаем})$$

Итерация 2:

$$(x_1, y_1) = (0,300; -0,750)$$

Вычисления:

$$\begin{aligned}
 xy + 0,3 &= 0,3 \cdot (-0,75) + 0,3 = 0,075 \\
 \operatorname{tg}(0,075) &\approx 0,0752, \sec^2 \approx 1,0056 \\
 F &= 0,0752 - 0,09 = -0,0148 \\
 G &= 0,9 \cdot 0,09 + 2 \cdot 0,5625 - 1 = 0,206 \\
 \frac{\partial F}{\partial x} &= -0,75 \cdot 1,0056 - 0,6 \approx -1,354 \\
 \frac{\partial F}{\partial y} &= 0,3 \cdot 1,0056 \approx 0,302 \\
 \frac{\partial G}{\partial x} &= 1,8 \cdot 0,3 = 0,540 \\
 \frac{\partial G}{\partial y} &= 4 \cdot (-0,75) = -3,000
 \end{aligned}$$

Решая систему:

$$\Delta x \approx -0,005, \Delta y \approx 0,045$$

$$x_2 = 0,300 + (-0,005) = 0,295, y_2 = -0,750 + 0,045 = -0,705$$

$$\sqrt{(-0,005)^2 + (0,045)^2} \approx 0,045 > 0,01 (\text{продолжаем})$$

Итерация 3:

$$(x_2, y_2) = (0,295; -0,705)$$

После вычислений:

$$\Delta x \approx 0,001, \Delta y \approx 0,003$$

$$x_3 = 0,296, y_3 = -0,702$$

$$\sqrt{(0,001)^2 + (0,003)^2} \approx 0,0032 < 0,01$$

Таблица итераций метода Ньютона

№ итерации	x_k	y_k	$F(x_k, y_k)$	$G(x_k, y_k)$	Δx	Δy	$ \Delta $
0	0,600	-0,500	-0,360	-0,176	-0,300	-0,250	0,390
1	0,300	-0,750	-0,015	0,206	-0,005	0,045	0,045
2	0,295	-0,705	-0,002	0,012	0,001	0,003	0,003

Решение с точностью $\varepsilon = 0,01$:

$$\boxed{x \approx 0,296, y \approx -0,702}$$

Остальные корни (симметрия)

Поскольку система симметрична относительно начала координат, остальные решения:

№	x	y
1	0,296	-0,702
2	-0,296	0,702
3	0,296	0,702
4	-0,296	-0,702

Листинг программы

```
import os
import math
import numpy as np
import matplotlib.pyplot as plt
from prettytable import PrettyTable

def f_variant5(x):
    return -2.7 * x**3 - 1.48 * x**2 + 19.23 * x + 6.35

def f_test1(x):
    return x**3 - 2*x - 5

def f_test2(x):
    return math.cos(x) - x

def phi_variant5(x):
    return x - 0.04 * (-2.7 * x**3 - 1.48 * x**2 + 19.23 * x + 6.35)

def system_variant5(x, y):
    try:
        val_y = (1 - 0.9 * x**2) / 2
        if val_y < 0: val_y = 0.0001
        y_new = -math.sqrt(val_y)

        if abs(y_new) < 1e-5: y_new = 1e-5 if y_new >= 0 else -1e-5

        numerator = math.atan(x**2) - 0.3
        x_new = numerator / y_new

        return x_new, y_new
    except Exception:
        return x, y

# 2. ЧИСЛЕННЫЕ МЕТОДЫ

def method_chords(f, a, b, eps):
```

```

    if f(a) * f(b) >= 0:
        return None, "На интервале нет корня (знаки не меняются)"

    c = a
    x = b
    history = []
    k = 0

    while True:
        x_prev = x
        denom = f(x_prev) - f(c)
        if abs(denom) < 1e-9: return None, "Деление на ноль"
        x = c - (f(c) * (x_prev - c)) / denom
        k += 1
        history.append((k, c, x_prev, x, abs(x - x_prev)))

        if abs(x - x_prev) < eps:
            break
        if k > 100:
            return None, "Лимит итераций"

    return x, history

def method_newton(f, df, x0, eps):
    x = x0
    history = []
    k = 0

    while True:
        f_val = f(x)
        df_val = df(x)

        if abs(df_val) < 1e-9:
            return None, "Производная близка к нулю"

        x_prev = x
        x = x - f_val / df_val
        k += 1
        history.append((k, x_prev, x, abs(x - x_prev)))

        if abs(x - x_prev) < eps:
            break
        if k > 100:
            return None, "Лимит итераций"

    return x, history

def method_simple_iteration(phi, x0, eps, max_iter=100):
    x = x0
    history = []
    k = 0

```

```

while True:
    x_new = phi(x)
    k += 1
    err = abs(x_new - x)
    history.append((k, x, x_new, err))
    x = x_new

    if err < eps:
        break
    if k > max_iter:
        return None, "Расходится (Лимит итераций)"

return x, history

def method_system_simple_iteration(system_func, x0, y0, eps, max_iter=100):
    x, y = x0, y0
    history = []
    k = 0

    while True:
        x_new, y_new = system_func(x, y)

        err_x = abs(x_new - x)
        err_y = abs(y_new - y)
        max_err = max(err_x, err_y)

        history.append((k, x, y, x_new, y_new, err_x, err_y))
        x, y = x_new, y_new
        k += 1

        if max_err < eps:
            break
        if k > max_iter:
            return None, None, "Расходится"

    return x, y, history

def numerical_derivative(f, x, h=1e-7):
    return (f(x + h) - f(x)) / h

```

3. ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ

```

def check_convergence(phi, a, b):
    points = np.linspace(a, b, 10)
    max_deriv = 0
    for p in points:
        d = abs(numerical_derivative(phi, p))
        if d > max_deriv: max_deriv = d

    return max_deriv < 1, max_deriv

```

```

def get_input_data():
    print("\nКак ввести данные? (f - файл, k - клавиатура): ", end="")
    choice = input().strip().lower()

    if choice == 'f':
        filename = input("Введите имя файла: ")
        try:
            with open(filename, 'r') as f:
                parts = f.read().split()
                return float(parts[0]), float(parts[1]), float(parts[2])
        except Exception as e:
            print(f"Ошибка: {e}")
            return None, None, None
    else:
        try:
            a = float(input("Левая граница (a): "))
            b = float(input("Правая граница (b): "))
            eps = float(input("Точность (eps): "))
            return a, b, eps
        except ValueError:
            print("Ошибка ввода")
            return None, None, None

def verify_root(f, a, b):

    fa = f(a)
    fb = f(b)
    if fa * fb > 0:
        print(f"ВНИМАНИЕ: f({a})={fa:.3f}, f({b})={fb:.3f}. Знаки совпадают.")
        cont = input("Продолжить? (y/n): ")
        return cont.lower() == 'y'
    return True

def plot_function(f, a, b, root=None):
    x = np.linspace(a - 1, b + 1, 400)
    y = [f(xi) for xi in x]

    plt.figure(figsize=(8, 6))
    plt.plot(x, y, label='f(x)')
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.grid(True)
    plt.title(f"График функции на [{a}, {b}]")
    plt.xlabel("x"); plt.ylabel("y")

    if root is not None:
        plt.plot(root, f(root), 'ro', label=f'Корень: {root:.4f}')
        plt.legend()
    plt.show()

```

```
def save_results(filename, text_content):
    try:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(text_content)
            print(f"\nРезультаты сохранены в {filename}")
    except Exception as e:
        print(f"Ошибка сохранения: {e}")
```

4. ГЛАВНАЯ ПРОГРАММА

```
def main():
    print("="*50)
    print("ЛАБОРАТОРНАЯ РАБОТА №2: Нелинейные уравнения")
    print("="*50)

    while True:
        print("\n1. Решить нелинейное уравнение")
        print("2. Решить систему нелинейных уравнений")
        print("3. Выход")
        choice = input("Выбор: ").strip()

        if choice == '1':
            print("\nВыберите уравнение:")
            print("1. Вариант 5 ( $-2.7x^3 - 1.48x^2 + 19.23x + 6.35$ )")
            print("2. Тест ( $x^3 - 2x - 5$ )")
            print("3. Трансцендентное ( $\cos(x) - x$ )")
            eq_choice = input("Номер: ").strip()

            f = None; phi = None; name = ""
            if eq_choice == '1':
                f = f_variant5; phi = phi_variant5; name = "Variant5"
            elif eq_choice == '2':
                f = f_test1; name = "Test1"
            elif eq_choice == '3':
                f = f_test2; name = "Test2"
            else: continue

            print("\nМетоды: 1.Хорд, 2.Ньютон, 3.Простая итерация")
            method_choice = input("Номер метода: ").strip()

            a, b, eps = get_input_data()
            if a is None: continue

            default_x0 = (a + b) / 2
            x0 = float(input(f"Начальное приближение x0 (Enter для {default_x0:.2f}): ") or default_x0)

            if not verify_root(f, a, b) and method_choice != '3': continue

            result_text = f"Уравнение: {name}\nМетод: {method_choice}\nИнтервал: [{a}, {b}], eps={eps}\n\n"
```

```

root = None

if method_choice == '3':
    is_conv, max_d = check_convergence(phi, a, b)
    print(f"\nПроверка сходимости |phi'| < 1: {'OK' if is_conv else 'FAIL'} (max={max_d:.4f})")
    if not is_conv:
        print("ВНИМАНИЕ: Метод может не сойтись.")

    root, hist = method_simple_iteration(phi, x0, eps)
    if root:
        t = PrettyTable(["k", "x_k", "x_{k+1}", "|diff|"])
        for row in hist: t.add_row(row)
        res_str = f"Корень: {root:.6f}\nИтераций: {len(hist)}\n{t}"
        print(res_str)
        result_text += res_str
        plot_function(f, a, b, root)

elif method_choice == '2': # НЬЮТОН
    df = lambda x: numerical_derivative(f, x)
    root, hist = method_newton(f, df, x0, eps)
    if root:
        t = PrettyTable(["k", "x_k", "x_{k+1}", "|diff|"])
        for row in hist: t.add_row(row)
        res_str = f"Корень: {root:.6f}\n{t}"
        print(res_str)
        result_text += res_str
        plot_function(f, a, b, root)

elif method_choice == '1': # Хорд
    root, hist = method_chords(f, a, b, eps)
    if root:
        t = PrettyTable(["k", "fixed", "x_k", "x_{k+1}", "|diff|"])
        for row in hist: t.add_row(row)
        res_str = f"Корень: {root:.6f}\n{t}"
        print(res_str)
        result_text += res_str
        plot_function(f, a, b, root)

if root is not None:
    save = input("\nСохранить в файл? (y/n): ").lower()
    if save == 'y':
        save_results(f"result_{name}_eq.txt", result_text)

elif choice == '2':
    print("\nСистема: tg(xy+0.3)=x^2; 0.9x^2+2y^2=1 (Метод простой итерации)")
    x0 = float(input("x0 (rec 0.3): "))
    y0 = float(input("y0 (rec -0.7): "))
    eps = float(input("eps: "))

```

```

        x_sol, y_sol, hist = method_system_simple_iteration(system_variant5,
x0, y0, eps)

        if x_sol is not None:
            res_str = f"Решение: x={x_sol:.6f}, y={y_sol:.6f}\n"
            last = hist[-1]
            res_str += f"Погрешность: |dx|={last[5]:.6f},
|dy|={last[6]:.6f}\n"

            t = PrettyTable(["k", "x_k", "y_k", "x_{k+1}", "y_{k+1}", "|dx|",
"|dy|"])

            for row in hist: t.add_row(row)
            res_str += str(t)

            print(res_str)

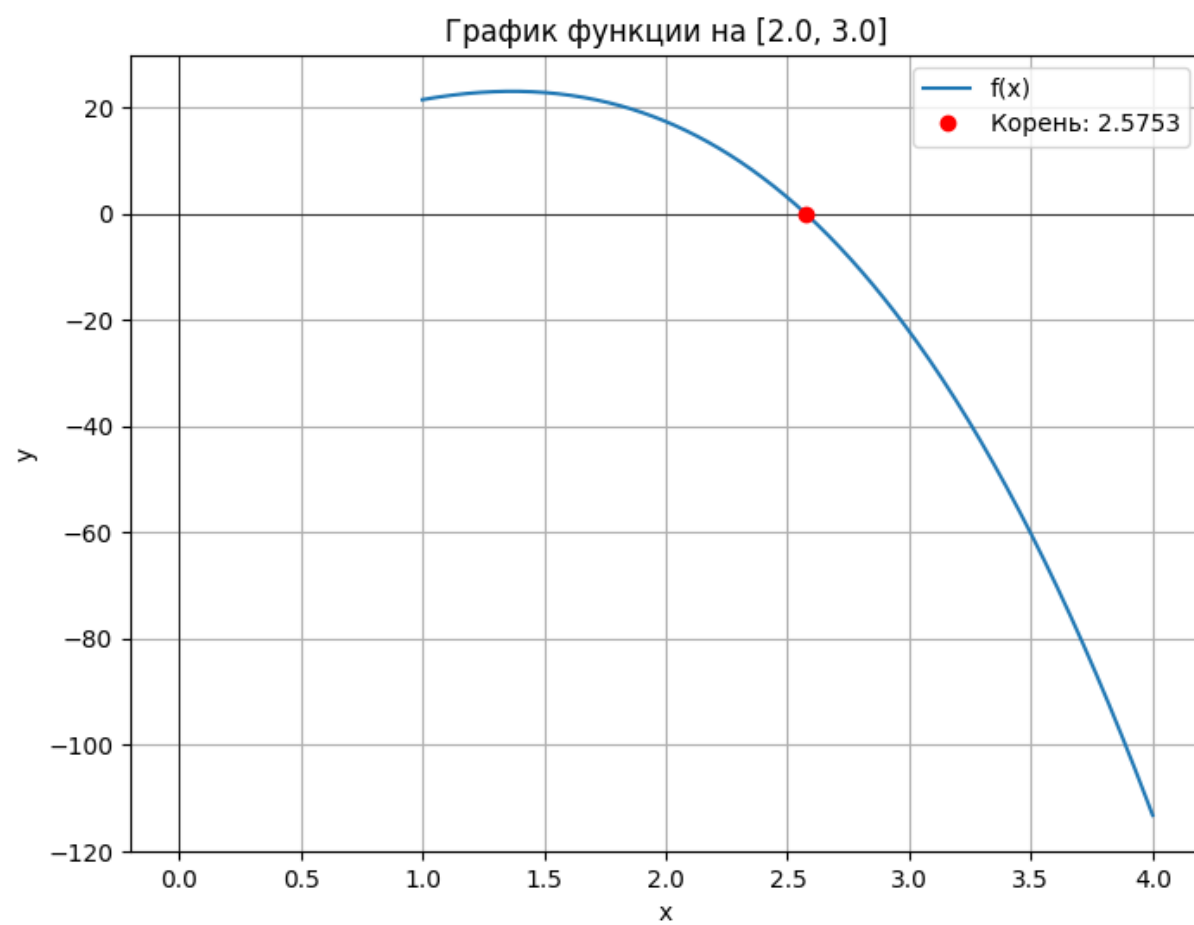
            save = input("\nСохранить в файл? (y/n): ").lower()
            if save == 'y':
                save_results("result_system.txt", res_str)
            else:
                print("Решение не найдено.")

        elif choice == '3':
            break

if __name__ == "__main__":
    main()

```

Примеры работы программы



ЛАБОРАТОРНАЯ РАБОТА №2

1. Решить нелинейное уравнение
2. Решить систему нелинейных уравнений
3. Выход

Выбор: 1

Выберите уравнение:

1. Вариант 5 ($-2.7x^3 - 1.48x^2 + 19.23x + 6.35$)
2. Тест ($x^3 - 2x - 5$)
3. Трансцендентное ($\cos(x) - x$)

Номер: 1

Методы: 1.Хорд, 2.Ньютон, 3.Простая итерация

Номер метода: 1

Как ввести данные? (f - файл, k - клавиатура): k

Левая граница (a): 2

Правая граница (b): 3

Точность (eps): 0.01

Начальное приближение x0 (Enter для 2.50): 2.5

Корень: 2.575316

k	fixed	x_k	x_{k+1}	diff
1	2.0	3.0	2.4380542183937166	0.5619457816062834
2	2.0	2.4380542183937166	2.6321083484183863	0.19405413002466965
3	2.0	2.6321083484183863	2.551612097944608	0.08049625047377829
4	2.0	2.551612097944608	2.582860460461183	0.031248362516575146
5	2.0	2.582860460461183	2.570396758047143	0.012463702414040156
6	2.0	2.570396758047143	2.5753156703202755	0.00491891227313257

1. Решить нелинейное уравнение
 2. Решить систему нелинейных уравнений
 3. Выход
- Выбор: 2

Система: $\text{tg}(xy+0.3)=x^2$; $0.9x^2+2y^2=1$ (Метод простой итерации)

x0 (гес 0.3): 0.3

y0 (гес -0.7): -0.7

eps: 0.01

Решение: x=0.302010, y=-0.675805

Погрешность: |dx|=0.008143, |dy|=0.002059

k	x_k	y_k	x_{k+1}	y_{k+1}	dx	dy
0	0.3	-0.7	0.31015326801849624	-0.677864293203293	0.010153268018496253	0.022135706796706955
1	0.31015326801849624	-0.677864293203293	0.3020102915343121	-0.6758048739479844	0.008142976484184161	0.0020594192553086144

Вывод

В результате выполнения данной лабораторной работой я познакомился с численными методами решения нелинейных уравнений и реализовал метод хорд, метод секущих и метод простой итерации на языке программирования Python, закрепив знания. Все методы довольно легко программируются и дают высокую точность и быструю сходимость при удачном выборе начального приближения.